



Università di Catania

DIPARTIMENTO DI MATEMATICA E INFORMATICA

SANTO LO BIANCO

1000061604

PROGETTO DATA MINING

**AUTOENCODER SU PROFILI DI ESPRESSIONE PER CALCOLO
DELLE CORRELAZIONI TRA GENI.**

INTRODUZIONE

Il presente progetto si pone l'obiettivo di analizzare la conservazione delle strutture geniche a seguito di tecniche di riduzione della dimensionalità. Le tecniche di riduzione sono:

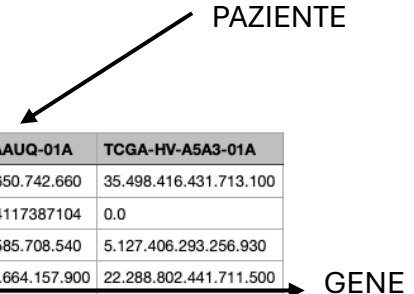
- **Principal Component Analysis(PCA)**: estrae i vettori ortogonali tra loro che preservano la varianza massima;
- **Auto-Encoder**: rete neurale addestrata a comprimere e ricostruire i dati. Questa compressione avviene attraverso un apprendimento dai dati di input.

Le matrici di partenza sono state scaricate da <https://xenabrowser.net/datapages/> e quelle che sono state richieste sono:

- **GDC TCGA Colon Cancer (COAD)** -> tumore del colon;
- **GDC TCGA Breast Cancer (BRCA)** -> tumore della mammella;
- **GDC TCGA Lung Adenocarcinoma (LUAD)** -> tumore al polmone
- **GDC TCGA Ovarian Cancer (OV)** -> tumore ovarico;
- **GDC TCGA Pancreatic Cancer (PAAD)** -> tumore del pancreas;
- **GDC TCGA Prostate Cancer (PRAD)** -> tumore della prostata.

Le matrici contengono dati di espressione genica, infatti:

- Pazienti risiedono nelle colonne;
- Geni risiedono nelle righe.



Ensembl_ID	TCGA-IB-A6UG-01A	TCGA-3A-A9IU-01A	TCGA-FB-A545-01A	TCGA-IB-AAUQ-01A	TCGA-HV-A5A3-01A
ENSG00000000003.15	3.746.022.478.160.720	4.085.594.615.025.430	30.094.905.029.474.500	2.983.221.650.742.660	35.498.416.431.713.100
ENSG00000000005.6	0.0	0.019203733973092663	0.0	0.10822334117387104	0.0
ENSG000000000419.13	437.873.355.926.538	4.787.730.209.055.910	4.874.905.139.385.390	4.596.708.585.708.540	5.127.406.293.256.930
ENSG000000000457.14	15.144.000.390.366.100	18.637.799.037.958.700	15.679.347.864.778.100	15.003.941.664.157.900	22.288.802.441.711.500
ENSG000000000460.17	0.7041637194539654	10.308.304.301.532.700	10.423.641.752.480.000	0.8666310806940011	11.963.552.428.827.400
ENSG000000000938.13	15.810.137.351.679.600	18.235.455.754.303.400	0.8004961475964861	1.827.737.743.733.840	26.405.515.215.374.000
ENSG000000000971.16	29.075.444.691.602.400	47.138.827.373.044	30.972.399.628.618.600	4.600.454.123.102.500	5.175.540.568.849.220

CARICAMENTO E PREPROCESSING DEI DATI(MAPPER)

La fase di pre-processing ha previsto la conversione degli identificativi Ensembl ID nei corrispondenti Gene Symbols. Tale mappatura è stata normalizzata rimuovendo le versioni(suffissi dopo il punto) e gestendo eventuali ridondanze.

Le classi prese in causa sono:

- **Preprocessor**: si occupa di effettuare una fase di preprocessing sulla matrice. I metodi utilizzati sono:
 - **_transpose_matrix**: metodo incaricato di invertire la matrice;
 - **_filter_constant_genes**: metodo incaricato della rimozione di geni la cui varianza è pari a 0 (geni costanti);
 - **_normalize**: applica lo StandardScaler alla matrice, ossia normalizza i dati ponendo media pari a 0 e deviazione standard pari a 1.

- **Mapper:** serve a tradurre gli Ensembl ID in nomi di leggibili. I metodi utilizzati sono:
 - **__init__**: data in input una matrice contenente l'ID del gene e il relativo nome crea un dizionario di mapping(id, nome);
 - **map**: prende in input una lista e sostituisce l'ID del gene con il nome corrispondente (preso dal dizionario) ove possibile. Nel caso in cui non si trovi la corrispondenza si mantiene l'ID del gene.

In uno spazio di soluzioni ampio, si è preferito utilizzare un file locale per effettuare la sostituzione degli ID con i nomi. In una fase iniziale, la sostituzione si era implementata mediante interrogazioni dinamiche a servizi esterni, come **MyGene**.

Tuttavia, data l'elevata dimensionalità del dataset e il numero consistente di richieste necessarie, tale approccio avrebbe introdotto un overhead computazionale significativo, oltre l'impossibilità di effettuare la sostituzione in assenza di connessione.

Oltretutto, i nomi dei geni potevano provenire da fonti diverse da quelle indicate; perciò, si è adottata una soluzione alternativa:

- è stato scaricato un dataset contenente le informazioni sui geni dal sito ufficiale: <https://www.ensembl.org/info/data/ftp/index.html>
- sono state rimosse tutte le informazioni irrilevanti ai fini del mapping;
- il dataset risultante è stato conservato in un file tsv, pronto per l'esecuzione.



*il lavoro sopracitato non è contenuto all'interno della cartella, in quanto estraneo alla consegna. Nonostante ciò, il file contenente queste informazioni è stato caricato all'interno del repository ed è accessibile mediante il link:

https://github.com/santolobianco-x/IDM-2025/blob/main/project/src/gene_mapping.tsv

La classe che carica la matrice processata è la classe **Dataset**. Essa nel caso in cui esiste il file corrispondente al path inserito caricherà il file e lo restituirà sotto forma di dataframe.

Qualora il file non sia presente, viene processato il dataset raw, salvato in memoria e infine caricato il DataFrame associato.

RIDUZIONE DEI DATI MEDIANTE AUTO-ENCODER E PCA

Una volta ottenuta la matrice di espressione pulita (che si è trasposta per una più facile lettura Pazienti x Geni), l'obiettivo è ridurre la dimensionalità dello spazio dei pazienti. Non vengono ridotti i geni, perché nella consegna è richiesto esplicitamente che nella fase successiva si dovrà calcolare la correlazione Gene x Gene. Questo passaggio è cruciale per catturare le caratteristiche latenti che descrivono ciascun gene.

Caricato il file si applica l'auto-encoder e la pca. Come nel caso della matrice preprocessata, quando si carica la matrice ridotta dall'auto-encoder, si verifica prima se già esiste una

versione calcolata in locale. La classe incaricata ad effettuare questo controllo è l'**Autoencoder_Reducer**, che attraverso i metodi:

- **getMatrixReduced** -> verifica se il file esiste al percorso passato. Nel caso in cui esista verrà caricato il file e restituito il dataframe, altrimenti si genera richiamando il metodo `_apply_autoencoder`;
- **_apply_autoencoder** -> passando i parametri ad un'istanza della classe **Autoencoder**, si richiama il metodo per il learning sui dati e infine viene restituita la matrice ridotta.

La classe citata **Autoencoder** è fondamentale per il secondo task, dato che ci permette di generare la matrice ridotta. Riportando il file sorgente:

```
def __init__(self, input_dim, encoding_dim=50, lr=0.001):
    self.input_dim = input_dim
    self.encoding_dim = encoding_dim

    if torch.backends.mps.is_available():
        self.device = torch.device("mps")
    elif torch.cuda.is_available():
        self.device = torch.device("cuda")
    else:
        self.device = torch.device("cpu")

    self.model = AENetwork(input_dim, encoding_dim).to(self.device)

    self.criterion = nn.MSELoss()

    self.optimizer = optim.Adam(self.model.parameters(), lr=lr)
```

Le due dimensioni indicate sono rispettivamente dimensioni in input (numero delle colonne) e dimensione restituita in output (codificata).

Attraverso torch, modulo di Pytorch, verifichiamo se per calcolare la matrice ridotta si può utilizzare:

- MPS -> gpu Apple;
- CUDA -> gpu NVIDIA;
- CPU;

Quando si dichiara il modello viene associata un'istanza della classe **AENetwork**, ossia la rete neurale vera e propria. Infine, viene scelta la **loss function** (funzione di perdita) ovvero l'**MSE** (Mean Square Error -> errore quadratico medio) e l'**optimizer Adam** (Adaptive Moment Estimation).

Adam è l'algoritmo che esegue la discesa del gradiente per aggiornare i parametri della rete neurale. Esso adatta la velocità di apprendimento per ogni singolo parametro ed è robusto e veloce a convergere. `lr` è il learning rate e stabilisce la velocità con cui si apprende, più grande è il valore e più la rete impara velocemente, con il rischio di non convergere ad un valore.

```
def fit(self, X, epochs=20, batch_size=32, verbose=1):
    X_tensor = torch.tensor(X, dtype=torch.float32)

    dataset = TensorDataset(X_tensor, X_tensor)
    dataloader = DataLoader(dataset,
                            batch_size=batch_size,
                            shuffle=True,
                            num_workers=0)

    self.model.train()
    for epoch in range(epochs):
        epoch_loss = 0.0
        for batch_in, batch_out in dataloader:
            batch_in, batch_out = batch_in.to(self.device), batch_out.to(self.device)

            self.optimizer.zero_grad()
            output = self.model(batch_in)
            loss = self.criterion(output, batch_out)
            loss.backward()
            self.optimizer.step()

            epoch_loss += loss.item()

        if verbose > 0:
            print(f"Epoch {epoch+1}/{epochs}, Loss: {epoch_loss/len(dataloader):.6f}")
```

I dati vengono trasformati in tensori, ossia delle strutture organizzate in più dimensioni. Questo permette quindi di lavorare parallelamente. Per ogni epoca la rete cerca di ricostruire i dati in input e si calcola la perdita rispetto all'originale. Attraverso la perdita vengono calcolati i parametri e infine attraverso Adam modificati ulteriormente. Tutto questo ripetuto per il numero di epoche specificato permette di costruire un modello abbastanza efficiente.

Effettuato il learning, attraverso la funzione `transform` si richiama nuovamente `encode` per comprimere i dati, che poi verranno restituiti sotto forma di **DataFrame**.

	AE1	AE2	AE3	AE4	AE5	AE6	AE7
TSPAN6	-0.1410308	0.7381705	2.9659214	-2.5905287	1.4343381	-0.3944907	2.174483
TNMD	-0.43150115	0.039663583	1.0780647	-2.197722	0.63369435	-0.33769393	0.9104971
DPM1	-0.285316	0.99923694	3.2963715	-1.3810264	0.6488251	-1.0807927	-0.2572558
SCYL3	0.8470898	-1.4799461	1.4195788	1.6600626	1.3080704	-0.79481435	-0.54551065
FIRRM	1.5221473	-0.10175627	0.44982463	0.37684795	1.9020917	-0.0068485886	-0.9338778
FGR	-2.8940687	0.77398515	0.60507196	2.1943438	-0.58960044	2.0498488	0.38611978
CFH	-2.746746	-0.32465807	1.0840293	1.4163463	-1.3571897	1.1492275	0.2837754
FUCA2	0.78898436	2.340091	1.859489	-1.4857911	0.5727074	0.54769087	-2.2991345
GCLC	1.8953972	1.979859	0.8039947	-1.9152555	0.4259024	-0.89217937	-1.0501499
NFYA	0.5754273	0.20123525	0.34672797	0.6114818	0.9219346	-0.11536409	0.3015115
STPG1	-0.19000334	-1.6842134	-0.40495294	-1.4163504	0.12490395	-2.8390098	1.2649889

**Ecco una porzione della matrice ridotta generata. Attraverso le componenti trovate si può ridurre lo spazio dei pazienti per ogni singolo gene.*

La vera riduzione avviene nella classe **AENetwork** citata prima. Essa è figlia della classe **nn.Module**, che contiene tutto il necessario per costruire le reti neurali, senza dover scrivere quindi a mano le formule matematiche.

```
class AENetwork(nn.Module):
    def __init__(self, input_dim, encoding_dim):
        super().__init__()

        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Linear(128, encoding_dim)
        )

        self.decoder = nn.Sequential(
            nn.Linear(encoding_dim, 128),
            nn.ReLU(),
            nn.Linear(128, input_dim)
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded

    def encode(self, x):
        return self.encoder(x)
```

L'encoder creato permette di:

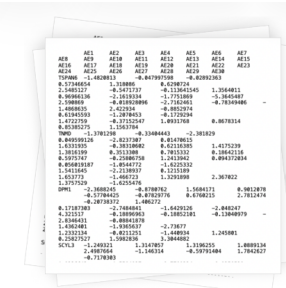
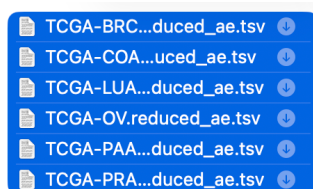
- ridurre la matrice dalla dimensione in input a 128, quindi, applicando l'operazione $y = A^T + b$ (matrice dei pesi);
- spegnere segnali deboli/negativi, introducendo non linearità (ReLU);
- ridurre ulteriormente la matrice da 128 dimensioni a quelle specificate in uscita.

Il decoder fa esattamente il contrario.

Quindi, a partire da un segnale ridotto si cerca di ripristinare l'originale.

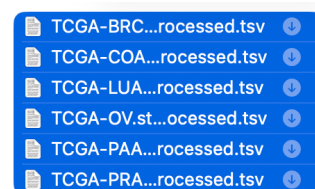
Dopo aver effettuato la compressione, i risultati ottenuti in termini di compressione sono ottimi, infatti, da centinaia di MB di file si arriva a pochi byte.

DIMENSIONE FILE COMPRESSI CON AE



6 elementi
6 documenti - 113 MB

DIMENSIONE ORIGINALE FILE



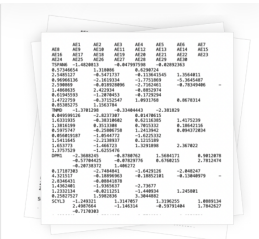
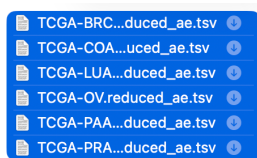
6 elementi
6 documenti - 2,26 GB

La seconda riduzione è stata ottenuta mediante pca, e la classe coinvolta è **PCA_reducer**. La PCA è una tecnica di riduzione dimensionale lineare che: trasforma i dati in nuove variabili ossia le componenti principali, mantenendo la massima varianza possibile ed eliminando ridondanze.

Come già visto, all'interno della classe si verifica se già esiste il file, se esiste si carica e si restituisce un dataframe. Nel caso contrario si calcola e si scelgono il numero di componenti necessarie per spiegare almeno il threshold (la soglia) della varianza.

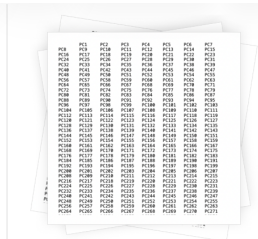
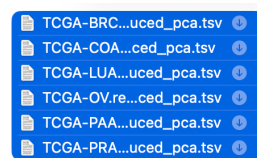
In questo caso anche se si hanno matrici ridotte, dato che si rimuove solo il rumore, il peso delle matrici in questione è comunque significativo. Mantenendo solo il 90% della varianza si ha:

DIMENSIONE FILE COMPRESSI CON AE



6 elementi
6 documenti **<113 MB**

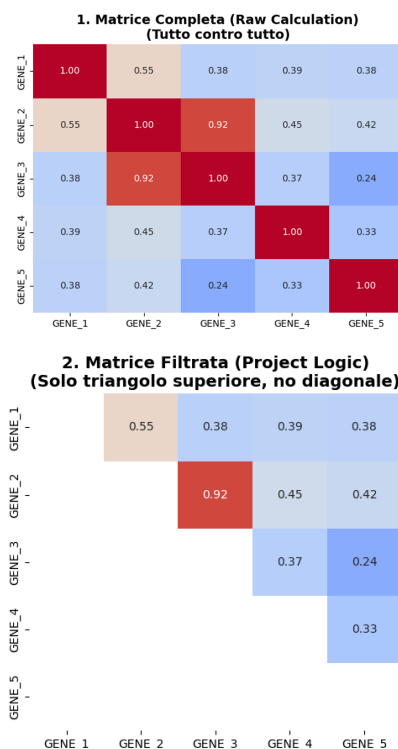
DIMENSIONE FILE RIDOTTI PCA



6 elementi
6 documenti **<1,31 GB**

CORRELAZIONE

L'obiettivo di principale di questo progetto, oltre che costruire l'auto-encoder, è il calcolo della correlazione. Ogni matrice tumorale contiene più di 60.000 mila geni per paziente e calcolare la correlazione gene x gene con la funzione standard potrebbe risultare insostenibile per le nostre risorse hardware.



Nonostante che si sia cambiato il tipo di dato (float64->float32), effettuare correlazione gene x gene produrrebbe una matrice contenente circa 3.600.000.000 valori.

La prima riduzione che si può effettuare sulla mole da calcolare è considerare che la matrice di correlazione è speculare, ossia che la triangolare superiore è uguale alla triangolare superiore.

Calcolando solo una delle due triangolari ridurrebbe il calcolo o ciò che si andrà a salvare della metà, infatti 3.600.000.000 -> 1.800.000.000.

Inoltre, si considera che il calcolo della correlazione di un gene con sé stesso restituirà 1.

In questo caso si è ridotta anche la dimensione del file.

Fatta questa premessa possiamo vedere cosa fa nel dettaglio la classe Correlation. La funzione d'interesse è la funzione **chunked_correlation** che accetta i seguenti parametri:

- **path** del file dove verranno scritti i risultati;
- **size** dei chunk(blocchi che verranno eseguiti);
- eventuale **threshold**(soglia);

Il punto forte della funzione è la suddivisione in blocchi. Questa scelta strutturale è stata intrapresa per evitare il collo di bottiglia che si crea nella RAM.

Infatti, anche se la CPU è libera e quindi può calcolare la correlazione, l'ammontare di tanti valori in RAM può causare un rallentamento in fase di calcolo.

Si normalizzano i valori sottraendo la media e dividendo per la deviazione standard, rendendo i calcoli molto più efficienti. Infine, viene applicata la maschera che prende solo la triangolare superiore e nel caso in cui c'è la soglia vengono presi tutti i valori sopra la soglia.

La scrittura dei risultati all'interno dei file avviene alla fine del calcolo della correlazione per ogni chunk, perché efficiente.

Se scrivessimo per ogni gene avremmo un overhead dato dai molti accessi in memoria, viceversa se scrivessimo una sola volta(tutte le correlazioni) avremmo il collo di bottiglia causato dalle molteplici informazioni in RAM.

COME VALUTARE LE CORRELAZIONI TROVATE??

La fase di valutazione ha lo scopo di determinare quanto le correlazioni calcolate sulle matrici ridotte siano fedeli rispetto a quelle ottenute dalla matrice originale.

A tal fine sono state sviluppate due classi:

- **ArtifactChecker**, che valuta l'errore sulla singola coppia di geni;
- **CorrelationAnalyzer**, che ingloba la classe ArtifactChecker e in generale fornisce una valutazione mediante statistiche e valutazioni.

L'idea alla base dell'ArtifactChecker è:

- considerare le coppie delle correlazioni trovate su matrici ridotte;
- confrontare i valori di correlazioni con le coppie calcolate sulla matrice originale;
- classificare come artefatto una correlazione quando la differenza tra il valore ottenuto nello spazio ridotto e quello originale supera una soglia di tolleranza ε .

Infine, si calcola la percentuale di artefatti presenti all'interno delle matrici di correlazioni generate. Questa metrica è fondamentale perché misura direttamente la distorsione introdotta dalla riduzione dimensionale.

La classe CorrelationAnalyzer, oltre alla percentuale di artefatti, calcola ulteriori metriche di valutazione. Esse sono:

- l'**indice di Jaccard**, che misura la sovrapposizione tra le correlazioni individuate con il metodo originale e quelle ottenute nello spazio ridotto;

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- l'**errore quadratico medio(RMSE)**;

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^n (x_i - y_i)^2}$$

- **correlazione di spearman**, ossia si misura quanto viene preservato l'ordinamento delle correlazioni. Si prendono i ranghi delle correlazioni e si confrontano;

$$\rho = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}$$

dove $d_i = R(x_i) - R(y_i)$.

L'analisi completa viene eseguita tramite il metodo **run_full_analysis()**, che applica in sequenza diverse fasi di valutazione per ciascun metodo di riduzione dimensionale.

In particolare, il metodo prevede:

- il confronto delle distribuzioni delle correlazioni(permette di capire se altera la struttura globale delle correlazioni);
- il calcolo delle metriche quantitative(citate prima);
- l'identificazione degli artefatti;
- la visualizzazione con scatter plot del confronto tra correlazioni originali e ridotte.

RISULTATI E DISCUSSIONE

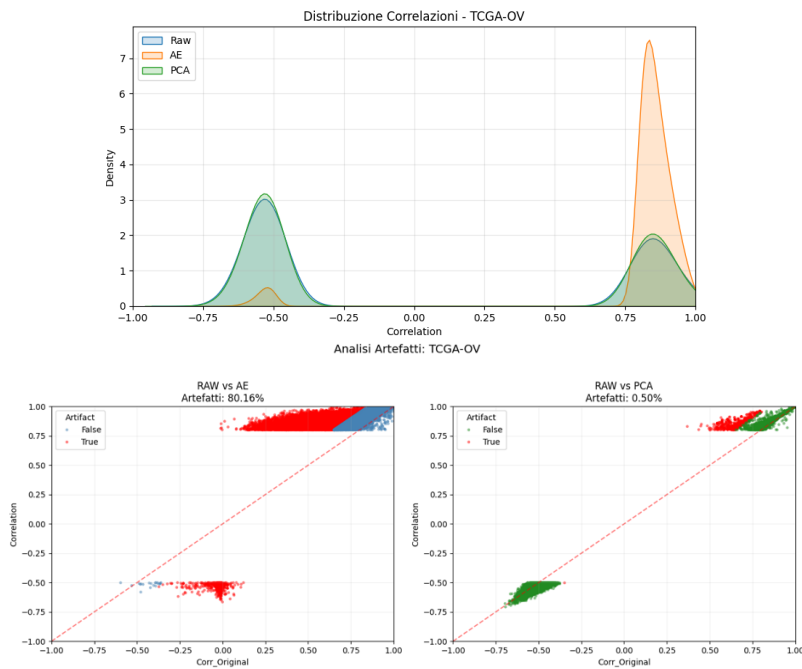
Gli iper-parametri che si sono utilizzati per l'acquisizione dei risultati sono:

- **encoding_dim** → dimensione dello spazio latente: 45;
- **epochs** → numero di volte che l'intero dataset viene passato attraverso la rete neurale: 25;
- **threshold** → varianza minima preservata dalla riduzione mediante PCA = 0.9;
- **pos_thresh** → soglia minima per correlazioni positive = 0.8;
- **neg_thresh** → soglia minima per correlazioni negative = -0,5(in minor presenza rispetto le correlazioni positive);
- **epsilon** → differenza massima accettabile tra correlazione nello spazio originale e nello spazio ridotto;

Tali valori sono stati selezionati empiricamente al fine di ottenere una riduzione dimensionale significativa senza introdurre una distorsione eccessiva nelle correlazioni(eccetto casi eccezionali).

Nei risultati ottenuti emergono chiaramente due casi estremi, che illustrano come la scelta della riduzione dimensionale possa influenzare la fedeltà delle correlazioni.

Esaminiamo il caso in cui l'auto-encoder genera risultati disastrosi. Questo estremo mostra che nel caso in cui ci troviamo in presenza di dati prettamente lineare la PCA riesce ad ottenere risultati migliori.

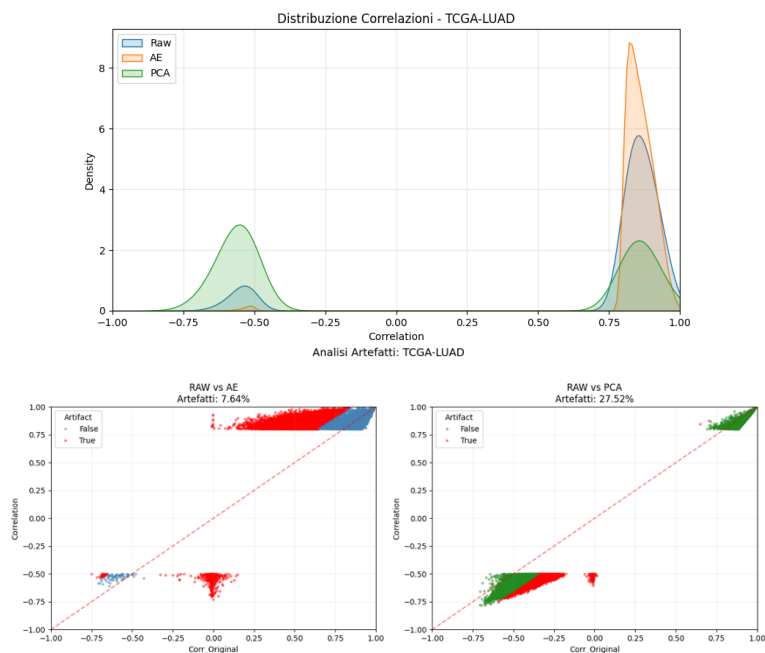


TCGA-OV(ovaio): l'auto-encoder produce un numero elevato di artefatti, circa l'80% delle correlazioni, alterando la struttura originaria.

La pca, invece, mantiene quasi intatte le correlazioni originali, con meno dello 0,5% di artefatti.

Questo evidenzia che in dataset quasi totalmente lineari, AE può generare "allucinazioni". In questo caso anche l'indice di Jaccard($=0,0679$) dell'auto-encoder mostra significative distorsioni.

Nonostante, il secondo estremo non sia completamente opposto ossia che la differenza non è significativa così tanto come nel primo caso, possiamo vedere come cambi la situazioni in presenza di dati non lineari.



TCGA-LUAD(polmone): la PCA introduce circa il 28% degli artefatti, mentre l'auto-encoder mantiene valori contenuti, ossia circa l'8%.

In questo caso quindi la capacità dell'AE di catturare le relazioni complesse non lineari, si rivela più efficace.

Nonostante ciò, la correlazione di Spearman mostra che l'ordine delle coppie è rispettato molto di più nella PCA(0,9414 vs 0,6716).

L'analisi condotta, mostra come l'effetto della riduzione dimensionale sulle correlazioni geniche non sia univoco.

L'auto-encoder mostra una sistematica tendenza a produrre correlazioni positive molto elevate. Questo comportamento è forse conseguenza della forte compressione nello spazio latente ridotto.

La PCA, che ricordiamo in questo caso preserva il 90% della varianza, riesce a mantenere un comportamento quasi costante, in presenza di dataset prettamente lineari.

Queste differenze emergono chiaramente dall'analisi quantitativa delle metriche.

Nel complesso, i risultati mostrano come non esistono metodi di riduzioni migliori di altri, ma la scelta è fortemente condizionata dal tipo di dataset e dall'obiettivo che si vuole raggiungere.